

Review that CanvasNet Fonts Subsystem Implementation is Correct

2026-10-02

Review that CanvasNet Fonts Subsystem Implementation is Correct

1. Introduction

1.1 Purpose

This document records the formal review of a set of project files.

1.2 Scope

This review covers the CanvasNet `Fonts` subsystem at the **subsystem level only**. The `Fonts` subsystem is entirely new this effort and comprises two units, `TrueTypeFont` and `SystemFontCatalog`, each of which has its own separate unit-level review-set (`CanvasNet-Fonts-TrueTypeFont`, `CanvasNet-Fonts-SystemFontCatalog`). Per the hierarchical-scope principle in `software-items.md`, unit-level implementation detail is excluded from this review; this review instead relies on the subsystem design document's description of what the two units provide, and verifies that the subsystem's own requirements, design, verification-design, bundled third-party asset documentation, and shared test infrastructure are internally correct, mutually consistent, and consistent with the surrounding system-level context.

Files under review (per `.reviewmark.yaml` review-set `CanvasNet-Fonts`):

- `docs/reqstream/canvas-net/fonts.yaml` (subsystem requirements)

Review that CanvasNet Fonts Subsystem Implementation is Correct

- `docs/design/canvas-net/fonts.md` (subsystem design)
- `docs/verification/canvas-net/fonts.md` (subsystem verification design)
- `test/DemaConsulting.CanvasNet.Tests/TestSupport/SyntheticFontBuilder.cs` (shared test fixture helper)
- `src/DemaConsulting.CanvasNet/Fonts/BundledFonts/README.md` (bundled third-party font licensing/provenance)

Context (not under review, used to understand intended role/scope only):

- `docs/design/canvas-net.md` (system-level design)
- `docs/design/introduction.md` (document conventions)
- `docs/reqstream/canvas-net.yaml` (system-level requirements)

This is the **first formal review ever conducted** for this review-set in this repository; the `reviews` evidence branch contained no prior evidence to compare against or supersede.

1.3 Outcomes

Each check must be recorded with one of the following outcomes:

Outcome	Meaning
Pass	The check was performed and the criterion is satisfied
Fail	The check was performed and the criterion is not satisfied
N/A	The check does not apply; justification is required

1.4 Review Details

Field	Value
Project	CanvasNet
Review ID	CanvasNet-Fonts
Review Title	Review that CanvasNet Fonts Subsystem Implementation is Correct
Fingerprint	01a08639bd6086fedfcda8c010419ecc3787df55fa6fce525b6f269a670fa46f
Review Date	2026-10-02

1.5 Reviewers

Name	Role	Organization	Signature	Date
Formal Review Agent (GitHub Copilot CLI)	Automated Reviewer	DEMA Consulting	(automated)	2026-10-02

1.6 Files Under Review

File
docs/reqstream/canvas-net/fonts.yaml
docs/design/canvas-net/fonts.md
docs/verification/canvas-net/fonts.md
test/DemaConsulting.CanvasNet.Tests/TestSupport/SyntheticFontBuilder.cs
src/DemaConsulting.CanvasNet/Fonts/BundledFonts/README.md

2. Review Checklist

2.1 Requirements Checks

Applicable: Yes

#	Check	Outcome	Justification
REQ-01	All requirements have a unique identifier	Pass	Every requirement in <code>fonts.yaml</code> has a unique <code>CanvasNet-Fonts-*</code> id; <code>dotnet reqstream --lint</code> confirms no duplicates.
REQ-02	All requirements are unambiguous	Pass	Requirement text is specific and scoped per capability (e.g. CFF outline decoding, TTC support, system font discovery).
REQ-03	All requirements are testable	Pass	Each requirement lists a <code>tests:</code> entry naming concrete automated tests.
REQ-04	All requirements are consistent	Pass	No contradictions found between sibling requirements.
REQ-05	All requirements are complete (no TBDs)	Pass	No TBD/placeholder markers found.
REQ-06	All requirements are verifiable	Fail	<code>CanvasNet-Fonts-TtcSupport</code> 's linked test <code>TrueTypeFont_RealTtcContainer_FaceZero_RendersGlyphGlyphOutlineAsVisibleI</code> does not exist anywhere in the test tree, so this requirement cannot actually be objectively confirmed met by the cited evidence. See Finding F2.
REQ-07	No compound requirements are present	Pass	Each requirement expresses a single testable capability.
REQ-08	No requirements are missing	Fail	<code>CanvasNet-Fonts-CffOutlineDecoding</code> and <code>CanvasNet-Fonts-Type1OutlineDecoding</code> omit four unit-level capabilities (<code>CffSeacComposition</code> , <code>CffCharsetResolution</code> , <code>CffWidthResolution</code> , <code>LoadType1C</code>) from their children lists, leaving those unit requirements orphaned/untraceable from the subsystem down to system root. See Finding F3.

2.2 Design Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
DES-01	Design documentation clearly describes the purpose of the component	Pass	<code>fonts.md</code> 's Purpose section clearly states the subsystem's responsibility (font parsing/matching for text rendering).
DES-02	Design documentation covers the necessary implementation details	Pass	At subsystem level, <code>fonts.md</code> appropriately describes each unit's public role without unit-internal detail, consistent with hierarchical scope.
DES-03	Design documentation describes how the code is interfaced	Pass	Units section documents <code>TrueTypeFont</code> and <code>SystemFontCatalog</code> public surfaces; Dependencies/Callers sections describe inbound/outbound interfaces.
DES-04	Design documentation describes the expected normal operation	Pass	Purpose/Units sections describe the normal font-loading/matching flow.
DES-05	Design documentation describes the expected error handling	N/A	At the subsystem level this repository's convention (consistent with <code>drawing.md</code> , <code>geometry.md</code>) defers error-handling detail to unit-level design documents; no subsystem-level error-handling claims are made that could be checked here.

2.3 Technical Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
DOC-01	Documentation is free of technical inaccuracies	Fail	<code>docs/verification/canvas-net/fonts.md</code> cites a non-existent test (<code>TrueTypeFont_RealTtcContainer_FaceZero_RendersGlyfGlyphOutlineAsVisib</code>) as acceptance-criteria proof for TTC support. See Finding F2. All other reviewed documentation, including <code>BundledFonts/README.md</code> 's licensing/provenance claims, empirically verified accurate (see Finding F1 narrative in section 3.1).
DOC-02	Documentation is consistent with the current implementation and requirements	Fail	Same phantom-test inconsistency as DOC-01; additionally the parent system-level design (<code>docs/design/canvas-net.md</code> , context) is stale with respect to the subsystem doc u review (omits <code>SystemFontCatalog</code> entirely). See Finding F4.
DOC-03	All referenced external documents and dependencies are correctly identified	Pass	<code>BundledFonts/README.md</code> correctly identifies Liberation Fonts v2.1.5 (SIL OFL 1.1) and Noto Project substitute fonts (SIL OFL 1.1); verified against actual embedded font binaries <code>.csproj</code> packaging — see Finding F1.
DOC-04	Documentation is free of spelling and grammar errors	Pass	<code>markdownlint-cli2</code> and <code>cspell</code> ran clean against all in-scope Markdown/YAML fi

2.4 Code Checks

Applicable: Yes (limited to the one source file in scope: the test-support helper; no production source files are in this review-set's paths)

#	Check	Outcome	Justification
CODE-01	Code conforms to the project coding standards and style guide	Pass	<code>SyntheticFontBuilder.cs</code> follows repository C# conventions (naming, XmlDoc, internal visibility).
CODE-02	No obvious resource leaks are present	Pass	Pure in-memory byte-array construction; no unmanaged resources, file handles, or streams held.
CODE-03	No hardcoded values are present that should be configurable	N/A	This is a test-fixture builder; the hardcoded SFNT/CFF/Type1 structural constants it uses are specification-mandated binary layout values, not configuration.
CODE-04	Each unit or function has a single, well-defined responsibility	Pass	Each of the 41 public/private methods builds exactly one font-table/structure.
CODE-05	Code is written at the appropriate level of abstraction	Pass	Stateless static-style builder methods per table, composed by instance methods; appropriate for a test fixture.
CODE-06	Code has an appropriate amount of extensibility for its context	Pass	New table/variant builders can be added following the existing pattern without touching unrelated code.

2.5 Logic Error Checks

Applicable: Yes (`SyntheticFontBuilder.cs` only)

#	Check	Outcome	Justification
LOGIC-01	Code does only what is intended	Pass	No side effects beyond returning constructed byte arrays; no static/shared mutable state across invocations.
LOGIC-02	All significant inputs and boundary conditions are handled correctly	Pass	Reviewed CFF/Type1 two-pass layout logic; offsets and lengths are computed consistently with documented invariants.
LOGIC-03	Concurrency and threading concerns are identified and addressed	N/A	Each <code>SyntheticFontBuilder</code> instance is independent, locally-scoped, single-threaded test fixture construction; no concurrency concern applies.

2.6 Error Handling & Logging Checks

Applicable: No

Not applicable: the files under review contain no production error-handling or logging logic.

`SyntheticFontBuilder.cs` is a test-only helper that, by design, is expected to throw on genuinely malformed builder usage (programmer error in a test), not to perform production-grade error handling.

2.7 Usability / Accessibility Checks

Applicable: Yes (documentation usability only)

#	Check	Outcome	Justification
USE-01	The feature or API is easy to use correctly	Pass	<code>SyntheticFontBuilder</code> 's fluent/explicit per-table methods are discoverable and self-describing for test authors.
USE-02	All public APIs are well documented	Pass	Every public and private member of <code>SyntheticFontBuilder.cs</code> carries WHY-focused XmlDoc, not just signature restatement.

2.8 Test Checks

Applicable: Yes (`SyntheticFontBuilder.cs` as shared test infrastructure)

#	Check	Outcome	Justification
TEST-01	Tests cover expected (happy-path) behavior	N/A	<code>SyntheticFontBuilder.cs</code> is test <i>infrastructure</i> (a fixture builder), not a test suite itself; it has no test cases of its own to assess for happy-path coverage.
TEST-02	Tests cover error conditions and boundary cases	N/A	Same reasoning as TEST-01; boundary/error-condition coverage is a property of the unit test suites that consume this helper, which are out of scope for this subsystem-level review.
TEST-03	Tests are independent and repeatable (no shared mutable state, no ordering dependency)	Pass	Each <code>SyntheticFontBuilder</code> instance owns its own <code>_tables</code> / <code>_sfntVersion</code> state; no static or shared mutable state exists that could create inter-test ordering dependencies.
TEST-04	Test names clearly describe the behavior being verified	N/A	<code>SyntheticFontBuilder.cs</code> contains builder methods, not test methods; naming of actual test methods is assessed under the respective unit review-sets.
TEST-05	New test cases are added for new functionality or defect fixes	N/A	Not assessable from the helper file alone at subsystem scope; however see Finding F2, which shows that for at least one subsystem requirement (<code>CanvasNet-Fonts-TtcSupport</code>) a planned test case was never actually implemented, even though it is cited as satisfying evidence.

2.9 Security Checks

Applicable: No

Not applicable: no authentication, authorization, or externally-sourced untrusted input handling is present in the files under review. (Bundled font binaries are reviewed under Technical Documentation / licensing checks, not security checks.)

2.10 Code Readability Checks

Applicable: Yes (`SyntheticFontBuilder.cs` only)

#	Check	Outcome	Justification
READ-01	Code is easy to understand	Pass	Clear separation of per-table builder methods; consistent structure throughout.
READ-02	Methods and functions are small enough to be easily understood	Pass	Methods are scoped to a single table/structure each.
READ-03	Symbols are well named	Pass	Method/field names directly mirror the SFNT/CFF/Type1 spec terminology they implement (<code>Head</code> , <code>Maxp</code> , <code>Hhea</code> , <code>Loca</code> , <code>Cff</code> , <code>Ttc</code> , etc.).
READ-04	Code is located in the correct place in the codebase	Pass	Resides under <code>test/.../TestSupport/</code> , correctly scoped as shared cross-subsystem test infrastructure rather than inside any one unit's test folder.
READ-05	Flow of control can be easily followed	Pass	Builder pattern with explicit two-pass layout clearly documented inline.
READ-06	Data flow is understandable	Pass	Byte-array construction flow is linear and documented at each step.
READ-07	Comments are provided where the code is non-obvious	Pass	Non-obvious binary-layout invariants (e.g., CFF two-pass offset computation) are explicitly explained.
READ-08	No debug artifacts or commented-out code have been left in the codebase	Pass	None found.

2.11 Requirements vs Documentation Checks

Applicable: Yes

#	Check	Outcome	Justification
REQDOC-01	All reviewed requirements are addressed in the general technical documentation	Pass	<code>fonts.md</code> (design) and <code>fonts.md</code> (verification) address all requirement groups in <code>fonts.yaml</code> .
REQDOC-02	No reviewed requirement is contradicted by the general technical documentation	Fail	The parent system-level design doc (<code>docs/design/canvas-net.md</code> , context) contradicts-by-omission the subsystem requirements/design: it describes the Fonts subsystem as exposing a single public unit (<code>TrueTypeFont</code>) and lists Fonts' only dependency as Geometry, while the in-scope requirements (<code>CanvasNet-Fonts-SystemFontDiscovery</code> et al.) and subsystem design document both describe a second public unit, <code>SystemFontCatalog</code> , with its own dependencies (<code>RuntimeInformation</code> , embedded <code>BundledFonts</code> resources). See Finding F4.

2.12 Requirements vs Implementation Checks

Applicable: No

Not applicable at this review's scope: per the hierarchical-scope principle, unit-level implementation (`TrueTypeFont` , `SystemFontCatalog` source code) is excluded from this subsystem-level review-set and is instead the subject of the separate unit-level review-sets `CanvasNet-Fonts-TrueTypeFont` and `CanvasNet-Fonts-SystemFontCatalog` . No production implementation source files are listed in this review-set's `paths` : .

2.13 Requirements vs Testing Checks

Applicable: Yes

#	Check	Outcome	Justification
REQTEST-01	Every requirement under review is covered by at least one test	Fail	<code>dotnet reqstream --enforce (root-tags system, quality)</code> reports <code>CanvasNet-Fonts-TtcSupport</code> and its ancestor <code>CanvasNet-Lib-FontsTtcSupport</code> as unsatisfied, because one of the two tests cited as its evidence does not exist. See Finding F2.
REQTEST-02	Tests verify the behavior described in each requirement	Fail	Same finding: the verification-design's Test Scenarios section for TTC support names a non-existent test as proof of glyph-flavored face-0 rendering, so the described behavior is not actually verified by any test. See Finding F2.

2.14 Code vs Design Documentation Checks

Applicable: No

Not applicable at this review's scope: the only source file under review, `SyntheticFontBuilder.cs`, is test infrastructure with no corresponding design document of its own (test helpers are not design-documented units per `design-documentation.md` / `software-items.md`); no production source code is in scope for this subsystem-level review.

3. Conclusion

3.1 Summary of Findings

#	Check	Finding
F1	DOC-03 (Pass — recorded for completeness given explicit review focus)	Bundled font licensing/provenance is accurate. <code>src/DemaConsulting.CanvasNet/Fonts/BundledFonts/README.md</code> claims the bundled fonts are Liberation Fonts v2.1.5 (SIL OFL 1.1) plus Noto Project substitute fonts for Symbol/ZapfDingbats fallback (also SIL OFL 1.1). This was independently verified by inspecting the actual embedded <code>.ttf</code> binaries with <code>fontTools</code> (all 12 Liberation fonts report exactly <code>Version 2.1.5</code> in their <code>name</code> table; the 3 Noto fonts report matching family names/versions) and by cross-checking <code>DemaConsulting.CanvasNet.csproj</code>'s <code>EmbeddedResource / Pack</code> entries, which embed and pack <code>OFL.txt</code> → <code>licenses\LiberationFonts-OFL.txt</code> and <code>NotoFonts-OFL.txt</code> → <code>licenses\NotoFonts-OFL.txt</code> exactly as the README describes. No inaccuracy found. Confidence: High (direct binary inspection plus <code>.csproj</code> cross-check). Low/informational note: the package's top-level <code>PackageLicenseExpression</code> is <code>MIT</code>; this is a legitimate and OFL-compliant pattern (bundled OFL license texts are redistributed alongside the binaries), but a consumer who inspects only the NuGet package's license expression — without opening the package contents or reading this README — would not learn that two sets of embedded binary assets carry a separate SIL OFL 1.1 license. Recommend (optional, not a compliance failure) a one-line pointer from the top-level repository <code>README.md</code> /package notes to this file.
F2	REQ-06, REQTEST-01, REQTEST-02, DOC-01, DOC-02	A cited test for TTC support does not exist. <code>docs/verification/canvas-net/fonts.md</code> 's Test Scenarios for <code>CanvasNet-Fonts-TtcSupport</code> name two tests as proof: <code>TrueTypeFont_RealTtcContainer_FaceOne_RendersCffGlyphOutlineAsVisibleInk</code> (exists, passes) and <code>TrueTypeFont_RealTtcContainer_FaceZero_RendersGlyfGlyphOutlineAsVisibleInk</code> (does not exist anywhere in <code>test/</code> — confirmed by exhaustive grep across the whole test tree). The same phantom test name is also listed in the in-scope subsystem requirement <code>CanvasNet-Fonts-TtcSupport</code> (<code>docs/reqstream/canvas-net/fonts.yaml</code>) and in the out-of-scope unit requirement <code>CanvasNet-Fonts-TrueTypeFont-LoadTtcExplicitFaceIndex</code> . Fresh TRX results (generated same day against a clean git tree) confirm the trace matrix (<code>dotnet reqstream --matrix</code>) shows 0 passed/0 failed executions for this test name, and as a direct consequence <code>dotnet reqstream --enforce (root-tags system,quality)</code> reports <code>CanvasNet-Lib-FontsTtcSupport</code> , <code>CanvasNet-Fonts-TtcSupport</code> , and <code>CanvasNet-Fonts-TrueTypeFont-LoadTtcExplicitFaceIndex</code> as unsatisfied/unproven. This means the glyf-outline (non-CFF) rendering path for TTC face 0 has no actual automated proof despite being documented as verified. Confidence: High (directly verified via exhaustive grep of <code>test/</code> plus <code>reqstream</code> trace-matrix evidence against a clean, same-day-built working tree).

#	Check	Finding
F3	REQ-08	Four unit requirements are orphaned from the subsystem's own decomposition. <code>dotnet reqstream --enforce (root-tags system,quality)</code> reports <code>CanvasNet-Fonts-TrueTypeFont-CffSeacComposition</code> , <code>CanvasNet-Fonts-TrueTypeFont-CffCharsetResolution</code> , <code>CanvasNet-Fonts-TrueTypeFont-CffWidthResolution</code> , and <code>CanvasNet-Fonts-TrueTypeFont-LoadType1C</code> as orphaned (unreachable from any root-tagged requirement). These exist as unit-level requirements (out-of-scope <code>true-type-font.yaml</code>) but the in-scope subsystem requirements file <code>docs/reqstream/canvas-net/fonts.yaml</code> does not list them in the <code>children:</code> of <code>CanvasNet-Fonts-CffOutlineDecoding</code> or <code>CanvasNet-Fonts-Type1OutlineDecoding</code> — i.e. the subsystem requirements file does not fully decompose/trace every capability the unit beneath it actually provides, per <code>requirements-principles.md</code> 's unidirectional-flow expectation that unit requirements trace up through their parent. Confidence: High (directly observed in both requirements files and in reqstream's own orphan-detection output).
F4	REQDOC-02, DES-02 (context cross-check)	Parent (context) system-level design doc is stale relative to the subsystem doc under review. <code>docs/design/canvas-net.md</code> (context, not itself under review) describes the Fonts subsystem's "External Interfaces" and dependency list as if <code>TrueTypeFont</code> were its only public unit and <code>Geometry</code> its only dependency. It contains zero mentions of <code>SystemFontCatalog</code> anywhere (confirmed by grep), even though the in-scope <code>docs/design/canvas-net/fonts.md</code> explicitly documents <code>SystemFontCatalog</code> as "the second public unit of the subsystem, added this phase," with its own public API (<code>Fonts</code> , <code>FindBestMatch</code> , <code>LoadBundledFallback</code>) and its own dependencies (<code>RuntimeInformation</code> , embedded <code>BundledFonts</code>). The in-scope requirements (<code>docs/reqstream/canvas-net.yaml</code> → <code>CanvasNet-Fonts-SystemFontDiscovery</code>) correctly trace down to the new unit, so only the parent design document is out of sync — it was not updated when <code>SystemFontCatalog</code> was added this phase. Because <code>canvas-net.md</code> is context rather than a file under review in this review-set, this is reported as an informational/process finding (not a Fail against a specific in-scope checklist row) but it should be corrected because it is read by this subsystem's own stakeholders as its authoritative parent description. Confidence: High (directly verified by grep: zero occurrences of "SystemFontCatalog" in <code>docs/design/canvas-net.md</code>).
F5	(Process/tooling, informational)	<code>.reviewmark.yaml</code> 's <code>needs-review</code> glob patterns silently exclude <code>BundledFonts/README.md</code> from automated elaboration/enforcement, despite it being explicitly listed in this review-set's (and <code>CanvasNet-Fonts-SystemFontCatalog</code> 's) <code>paths:</code> . The root <code>needs-review</code> patterns match <code>README.md</code> only at the repository root, <code>**/*.cs</code> , and various <code>docs/**/*.md</code> / <code>.yaml</code> patterns — none match an arbitrary non-root <code>README.md</code> under <code>src/</code> . Consequently

#	Check	Finding
		<code>dotnet reviewmark --elaborate CanvasNet-Fonts</code> lists only 4 files (omitting the README), and <code>dotnet reviewmark --plan --enforce</code> does not fingerprint or track this file as part of any review-set's compliance evidence, even though it was explicitly singled out in this review's commissioning instructions as a point of particular licensing concern. <code>dotnet reviewmark --lint</code> does not flag this gap. The same glob gap also silently drops <code>src/DemaConsulting.CanvasNet/DemaConsulting.CanvasNet.csproj</code> from <code>CanvasNet-Fonts-SystemFontCatalog</code>'s elaborated file list for the identical reason. This was reviewed manually per the task's explicit instruction (see Finding F1), but the automated tooling's enforcement pipeline does not currently cover it. Recommendation: add a <code>needs-review</code> pattern covering non-root <code>README.md</code> files (and/or <code>.csproj</code> files) that are referenced by any review-set's <code>paths: .</code> <code>.reviewmark.yaml</code> is a protected configuration file and was not modified by this review. Confidence: High (directly reproduced: elaborate output verified to list only 4 of the 5 configured files; pattern list inspected directly).
F6	(Informational, out of scope)	A repository-wide <code>dotnet reviewmark --plan --enforce</code> run reports 8 globally uncovered files, three of which are in the Fonts area: <code>src/DemaConsulting.CanvasNet/Fonts/CffStandardEncoding.cs</code>, <code>test/DemaConsulting.CanvasNet.Tests/Fonts/BundledNotoFontsTests.cs</code>, and <code>test/DemaConsulting.CanvasNet.Tests/Fonts/CffStandardEncodingTests.cs</code>. These are unit-level files, correctly out of scope for this subsystem-level review per the hierarchical-scope principle; flagged here only for the orchestrator's awareness, to be addressed (if appropriate) under the relevant unit-level review-sets or a <code>.reviewmark.yaml</code> coverage update.

3.2 Overall Outcome

Overall Outcome: Fail

The `Fonts` subsystem's design, requirements, and verification documentation are substantially well-constructed (clear subsystem design following the repository's established Purpose/Units/Dependencies/Callers convention, no spelling/lint issues, and — of particular note given this review's explicit focus — fully accurate bundled third-party font licensing and provenance claims, empirically verified against the actual embedded binaries). The shared test-fixture helper `SyntheticFontBuilder.cs` is well-designed, well-documented, and trustworthy shared test infrastructure with no significant findings.

However, the review identifies two substantive, objectively-verified compliance gaps that must be resolved before this review-set can be approved:

1. **(F2)** The verification design and subsystem requirements both cite a test (`TrueTypeFont_RealTtcContainer_FaceZero_RendersGlyfGlyphOutlineAsVisibleInk`) that does not exist, meaning the TTC-support requirement chain (`CanvasNet-Lib-FontsTtcSupport` → `CanvasNet-Fonts-TtcSupport` → `CanvasNet-Fonts-TrueTypeFont-LoadTtcExplicitFaceIndex`) is not actually proven satisfied as documented.
2. **(F3)** The subsystem requirements file fails to decompose four unit-level CFF/Type1 capabilities into its `children:` lists, leaving them orphaned from the system/quality root tags.

Follow-up actions required before approval:

- Either implement the missing TTC face-0 glyf-rendering test, or correct the verification design and requirements to cite only tests that actually exist (F2).
- Add the four missing unit requirement IDs to the appropriate `children:` lists in `docs/reqstream/canvas-net/fonts.yaml` (F3).
- Update `docs/design/canvas-net.md` (parent/system-level design) to document `SystemFontCatalog` as the Fonts subsystem's second public unit (F4).
- Consider adding a `needs-review` glob pattern so `BundledFonts/README.md` (and similarly-situated non-root README/ `.csproj` files referenced by `paths:`) are not silently excluded from ReviewMark's automated elaboration and enforcement (F5).

No corrections were made by this review; all findings are reported for the orchestrator to act on, per this review's read-only audit mandate.